

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"Сибирский государственный университет телекоммуникаций и
информатики"

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЁТ
о практической работе №1
по дисциплине
"Основы систем мобильной связи"

по теме:
ВРЕМЕННАЯ И ЧАСТОТНАЯ ФОРМЫ СИГНАЛОВ.
ПРЕОБРАЗОВАНИЯ ФУРЬЕ. ДИСКРЕТИЗАЦИЯ СИГНАЛОВ

Студент:
Группа ИКС-433

Воробьёв Д. А.

Преподаватель:
заведующий кафедрой

Дроздова В. Г.

Новосибирск 2026 г.

СОДЕРЖАНИЕ

1	ПРАКТИЧЕСКАЯ РАБОТА №1. ВРЕМЕННАЯ И ЧАСТОТНАЯ ФОРМЫ СИГНАЛОВ. ПРЕОБРАЗОВАНИЯ ФУРЬЕ. ДИСКРЕТИЗАЦИЯ СИГНАЛОВ	3
1.1	Цель и задачи работы	3
1.2	Краткие теоретические сведения	3
1.2.1	Дискретизация и теорема Котельникова	3
1.2.2	Квантование и АЦП	4
1.2.3	Дискретное преобразование Фурье	4
1.2.4	Быстрое преобразование Фурье	5
1.2.5	Ширина спектра	5
1.3	Исходные данные	5
1.4	Ход работы	7
1.4.1	Задание 1	7
1.4.2	Задание 2	8
1.4.3	Задание 3	9
1.4.4	Задание 4	10
1.4.5	Задание 5	12
1.4.6	Задание 6	17
1.4.7	Задание 7	19
1.4.8	Задание 8	23
1.4.9	Задание 9	24
1.4.10	Задание 10	24
1.4.11	Задание 11	25
1.4.12	Задание 12	27
1.4.13	Задание 13	31
1.5	Ответы на контрольные вопросы	36
1.6	Вывод	37
1.7	Ссылка на репозиторий с кодом	38

1 ПРАКТИЧЕСКАЯ РАБОТА №1. ВРЕМЕННАЯ И ЧАСТОТНАЯ ФОРМЫ СИГНАЛОВ. ПРЕОБРАЗОВАНИЯ ФУРЬЕ. ДИСКРЕТИЗАЦИЯ СИГНАЛОВ

1.1 Цель и задачи работы

Цель работы: изучить влияние частоты дискретизации и разрядности АЦП на частотные характеристики сигнала, освоить дискретные преобразования Фурье для переходов между временным и частотным представлениями сигналов.

Задачи работы:

- сгенерировать и визуализировать непрерывный сигнал по варианту, определить максимальную частоту в его спектре и минимальную частоту дискретизации по теореме Котельникова;
- оцифровать сигнал, выполнить прямое дискретное преобразование Фурье, оценить ширину спектра и объём памяти для хранения отсчётов;
- восстановить сигнал по отсчётам и оценить сходство с оригиналом, повторить эксперимент при частоте дискретизации, увеличенной в 4 раза;
- записать голос, определить его максимальную частоту и частоту дискретизации записи, оценить влияние неверно выбранной частоты дискретизации на звук и на спектр;
- оценить влияние разрядности АЦП на спектр сигнала и среднюю ошибку квантования.

1.2 Краткие теоретические сведения

1.2.1 Дискретизация и теорема Котельникова

Чтобы обрабатывать сигнал на компьютере, непрерывную функцию $x(t)$ заменяют набором её значений (отсчётов) в моменты времени, кратные периоду дискретизации $T_s = 1/f_s$:

$$x[n] = x(nT_s) = x\left(\frac{n}{f_s}\right), \quad n = 0, 1, 2, \dots$$

Теорема Котельникова: сигнал со спектром, ограниченным частотой f_{max} , можно без потерь восстановить по его отсчетам, если частота дискретизации строго больше удвоенной максимальной частоты:

$$f_s > 2f_{max}.$$

Если условие не выполняется, высокие частоты сигнала "складываются" с низкими, и исходный сигнал восстановить нельзя.

1.2.2 Квантование и АЦП

Аналого-цифровой преобразователь (АЦП) выполняет дискретизацию по времени и квантование по уровню: амплитуду каждого отсчета округляют до одного из 2^n возможных значений, где n - разрядность АЦП. Шаг квантования при размахе сигнала от x_{min} до x_{max} равен

$$\Delta = \frac{x_{max} - x_{min}}{2^n - 1},$$

а ошибка квантования (разница между исходным и округлённым значением) не превышает $\Delta/2$. Чем больше разрядов у АЦП, тем меньше ошибка квантования. Для хранения одного отсчета нужно n бит, поэтому увеличение разрядности и частоты дискретизации увеличивает и объём памяти.

1.2.3 Дискретное преобразование Фурье

Любой сигнал можно представить суммой гармоник с разными частотами, амплитудами и фазами. Прямое преобразование Фурье (ППФ) переводит временное представление сигнала в частотное, обратное преобразование Фурье (ОПФ) выполняет обратный переход. Для дискретного сигнала из N отсчетов прямое преобразование имеет вид (1.8):

$$F(x) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi \frac{xn}{N}},$$

а обратное (1.9):

$$f(n) = \frac{1}{N} \sum_{x=0}^{N-1} F(x) e^{j2\pi \frac{xn}{N}}.$$

Результат ППФ - массив из N комплексных чисел. Номер элемента k соответствует частоте $f_k = kf_s/N$, амплитуда гармоники равна $|F(k)|/N$, фаза - аргументу комплексного числа. Для вещественного сигнала вторая половина спектра является зеркальным отражением первой, поэтому ширину спектра оценивают по первой половине (до $N/2$).

1.2.4 Быстрое преобразование Фурье

Прямое вычисление по формуле требует порядка N^2 операций. Быстрое преобразование Фурье (БПФ) группирует слагаемые с одинаковыми множителями и вычисляет то же преобразование за порядка $N \log_2 N$ операций, при этом длина входного массива обычно равна $N = 2^k$. БПФ используется в модемах мобильных устройств, а в работе - для спектра длинной записи голоса.

1.2.5 Ширина спектра

Шириной спектра называют диапазон частот, в котором сигнал имеет заметную амплитуду. Из-за ошибок округления отсутствующие в сигнале частоты получаются не точно нулевыми, поэтому значимыми считают частоты, амплитуда которых превышает заданный порог.

1.3 Исходные данные

Вариант 5. Непрерывная периодическая функция:

$$y(t) = \cos\left(2\pi ft + \frac{\pi}{3}\right) + \sin(4\pi ft), \quad f = 4.$$

Сигнал рассматривается на интервале длительностью 1 с, для имитации непрерывного сигнала используется сетка в 1024 отсчёта на секунду. Запись голоса: файл `voice.wav`, моно, частота дискретизации 44100 Гц, длительность 6,14 с.

Расчёты выполнены на языке Python с использованием библиотек NumPy, SciPy, Matplotlib и icecream. Ниже приведены подключаемые функции проекта и параметры сетки времени.

```
1 import sys
2 from pathlib import Path
3
4 sys.path.insert(0, str(Path.cwd().parent))
5
6 from icecream import ic
7
8 import numpy as np
9 import utils.debug
10 from matplotlib.axes import Axes
11
12 from utils.complex_numbers import to_polar, to_rect
13 from utils.harmonic import (
14     get_harmonic_oscillation,
15     get_period,
16     get_phase_time_shift,
17     get_phase_at,
18 )
19 from utils.spectrum import get_spectrum, get_spectrum_width
20 from utils.fourier import discrete_fourier_transform
21 from utils.quantize import quantize, quantize_np
22 from utils.plotting import show_multi_plot, show_plot
23 from utils.sampling import get_linspace
24 from scipy.io import wavfile
25 from IPython.display import Audio
```

Листинг 1.1 - Подключение библиотек и функций проекта

```
1 sampling_frequency = 1024
2 start = 0
3 stop = 1
4 t = get_linspace(start=start, stop=stop, fs=sampling_frequency)
```

Листинг 1.2 - Параметры времени и сетка отсчётов

1.4 Ход работы

1.4.1 Задание 1

Необходимо сгенерировать и визуализировать (вывести на график) непрерывный сигнал, в соответствии с вариантом (таблица 1.1). Номер варианта - порядковый номер в журнале группы.

```
1 frequency = 4
2
3 ic(frequency)
4 ic(sampling_frequency)
5
6 y_1 = get_harmonic_oscillation(1, frequency, np.pi/3, wave=np.cos)(t)
7 y_2 = get_harmonic_oscillation(1, 2*frequency, 0, wave=np.sin)(t)
8 y = y_1 + y_2 # 5 вариант
9
10 show_plot(
11     t=t,
12     func_val=y,
13     title="cos(2*pi*f*t + pi/3) + sin(4*pi*f*t)",
14 )
```

Листинг 1.3 - Генерация и построение непрерывного сигнала

```
ic| frequency: 4
ic| sampling_frequency: 1024
```

Листинг 1.4 - Результат выполнения: генерация и построение непрерывного сигнала

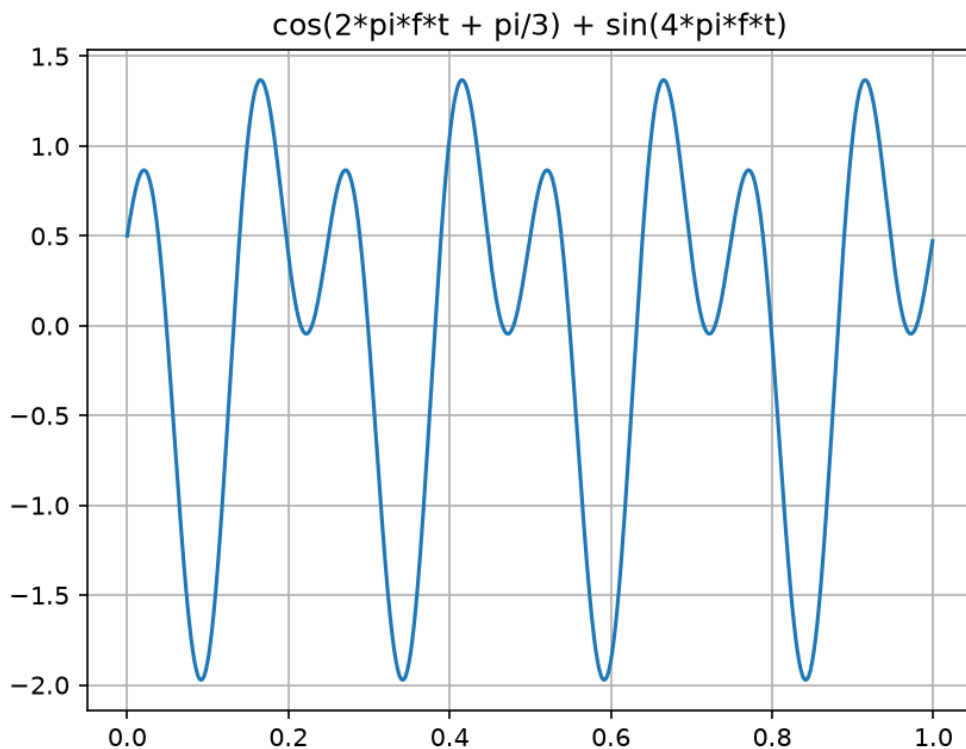


Рисунок 1 - Непрерывный сигнал варианта 5

Вывод

Сигнал состоит из двух гармоник и повторяется каждую 1/4 секунды (период медленной гармоники 4 Гц), на графике это 4 одинаковых периода. Размах сигнала примерно от -2 до 1,4.

1.4.2 Задание 2

Определить максимальную частоту в спектре данного сигнала.

Расчёт. Сигнал (варианта 5) состоит из двух гармоник:

$$y(t) = \cos(2\pi f t + \frac{\pi}{3}) + \sin(4\pi f t), \quad f = 4 \text{ Гц}$$

Частота каждой гармоники:

- $\cos(2\pi f t + \frac{\pi}{3})$: частота $f_1 = f = 4 \text{ Гц}$;
- $\sin(4\pi f t) = \sin(2\pi \cdot 2f \cdot t)$: частота $f_2 = 2f = 8 \text{ Гц}$.

Спектр сигнала состоит из этих двух частот, поэтому максимальная частота - наибольшая из них:

$$f_{max} = \max(f_1, f_2) = 2f = 2 \cdot 4 = 8 \text{ Гц}$$

```
1 max_freq = frequency * 2
2 ic(max_freq);
```

Листинг 1.5 - Максимальная частота сигнала

```
ic| max_freq: 8
```

Листинг 1.6 - Результат выполнения: максимальная частота сигнала

1.4.3 Задание 3

Определить минимальную необходимую частоту дискретизации полученного сигнала (теорема Котельникова).

```
1 # min_freq_kotelnikov = max_freq * 2
2 min_freq_kotelnikov = max_freq * 4
3
4 ic(max_freq)
5 ic(min_freq_kotelnikov);
```

Листинг 1.7 - Частота дискретизации по Котельникову

```
ic| max_freq: 8
ic| min_freq_kotelnikov: 32
```

Листинг 1.8 - Результат выполнения: частота дискретизации по Котельникову

Примечание. По теореме Котельникова f_s должна быть *строго больше* $2f_{max}$. При $f_s = 2f_{max} = 16$ Гц отсчёты восьмигерцовой гармоники $\sin(4\pi ft)$ попадают ровно в нули синуса, и она теряется.

Брать произвольное $f_s > 16$ Гц (например, 17 или 20) неудобно: сетка 1024 точки на 1 секунду не делится на такое число нацело, поэтому шаг `step` получается дробным и отсчёты получаются неравномерными.

Поэтому выбрано $f_s = 4f_{max} = 32$ Гц: это ближайшая степень двойки выше минимума, она делит 1024 нацело (шаг 32), отсчёты равномерны.

1.4.4 Задание 4

Оцифровать сигнал с полученной частотой дискретизации, выбрав требуемое число отсчетов сигнала на длительности 1 секунда, сохранить полученные значения в массив, пока не озадачиваясь разрядностью АЦП, просто выбранные с частотой дискретизации значения с теми уровнями, которые имеет функция в полученных точках.

```
1 step = len(y) // int(min_freq_kotelnikov * (stop - start))
2
3 # прореживая по шагу step
4 y_sampled = y[::step]
5 t_sampled = t[::step]
6
7 ic(y_sampled)
8 ic(t_sampled);
```

Листинг 1.9 - Дискретизация сигнала

```
ic| y_sampled: array([ 0.5          ,  0.74118095, -0.8660254 ,
-1.96592583, -0.5          ,
1.25881905,  0.8660254 , -0.03407417,  0.5
,  0.74118095,
-0.8660254 , -1.96592583, -0.5          ,
1.25881905,  0.8660254 ,
-0.03407417,  0.5          ,  0.74118095, -0.8660254
, -1.96592583,
-0.5          ,  1.25881905,  0.8660254 ,
-0.03407417,  0.5          ,
0.74118095, -0.8660254 , -1.96592583, -0.5
,  1.25881905,
0.8660254 , -0.03407417])
ic| t_sampled: array([0.          , 0.03125, 0.0625 , 0.09375, 0.125 ,
0.15625, 0.1875 ,
0.21875, 0.25   , 0.28125, 0.3125 , 0.34375,
0.375 , 0.40625,
0.4375 , 0.46875, 0.5     , 0.53125, 0.5625 ,
0.59375, 0.625 ,
0.65625, 0.6875 , 0.71875, 0.75   , 0.78125,
0.8125 , 0.84375,
0.875 , 0.90625, 0.9375 , 0.96875])
```

Листинг 1.10 - Результат выполнения: дискретизация сигнала

```

1 def draw_samples(ax: Axes) -> None:
2     ax.stem(t_sampled, y_sampled, linefmt="r-", markerfmt="ro", basefmt
3         =" ")
4
5 show_plot(
6     t=t,
7     func_val=y,
8     title="Сигнал и отсчеты",
9     on_axes=draw_samples,
10 )

```

Листинг 1.11 - Построение сигнала и отсчётов

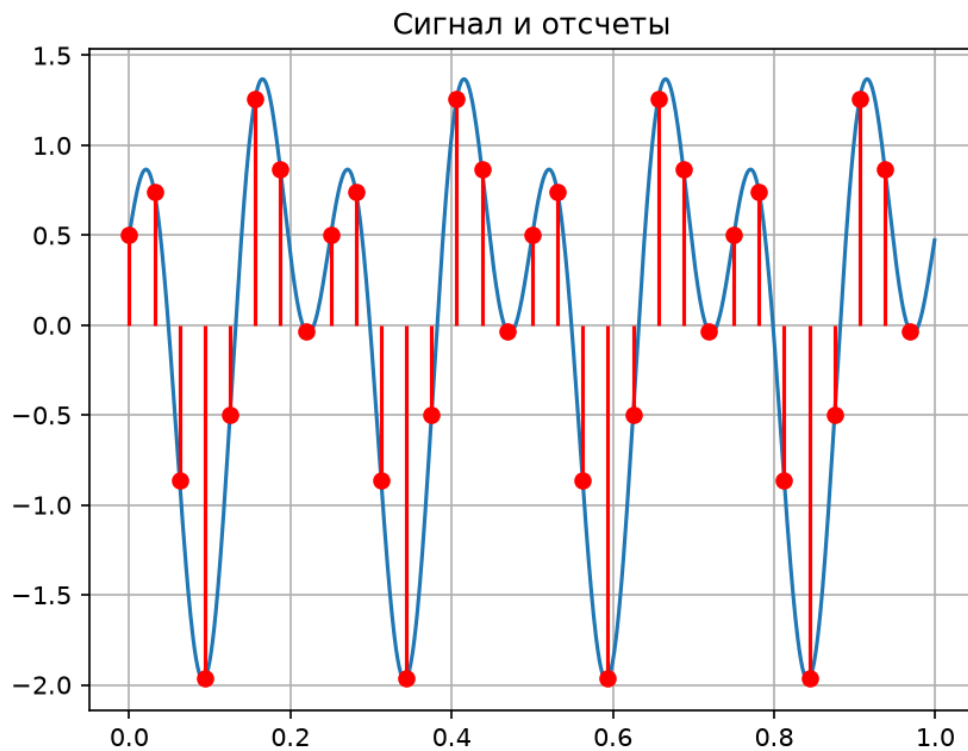


Рисунок 2 - Сигнал и отсчёты, $f_s = 32$ Гц

```

1 show_plot(
2     t=t_sampled,
3     func_val=y_sampled,
4     title=f"частота по Котельникову = {min_freq_kotelnikov}",
5 )

```

Листинг 1.12 - Построение сигнала по отсчётам



Рисунок 3 - Сигнал по отсчётам, $f_s = 32$ Гц

Вывод

Взяли каждый 32-й отсчёт из 1024, получилось 32 отсчёта на секунду, то есть частота дискретизации 32 Гц. Обе гармоники сигнала по ним различимы.

1.4.5 Задание 5

Выполнить прямое дискретное преобразование Фурье для массива временных отсчетов сигнала и оценить ширину данного спектра. А также объем памяти, требуемый для хранения данного массива (тип переменных в массиве - на ваше усмотрение, float, int и пр.).

Что делает преобразование Фурье

Мы смотрим на сигнал и спрашиваем: Из каких гармоник он состоит? Для этого мы используем прямое преобразование Фурье по формуле:

$$F(x) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi \frac{xn}{N}},$$

где N - размерность преобразования Фурье, $F(x)$ - сигнал в частотной области и $f(n)$ - дискретизированный сигнал во временной области.

Листинг функции `utils/fourier.py`:

```
1 def discrete_fourier_transform(x: np.ndarray) -> np.ndarray:
2     N = len(x)
3     X = []
4     for k in range(N):
5         s = 0
6         for n in range(N):
7             s += x[n] * np.exp(-2j * np.pi * k * n / N)
8         X.append(s)
9
10    return np.array(X)
```

Листинг 1.13 - Функция прямого ДПФ (`utils/fourier.py`)

Что она возвращает. Массив из N комплексных чисел (столько же, сколько отсчётов на входе). Число номер k относится к частоте номер k . Комплексное число хранит сразу две вещи: амплитуду и фазу.

Как достать нужное:

Что нужно	Как получить	В коде
Частота k -й гармоники	номер умножить на шаг частоты	<code>k * fs / N</code>
Амплитуда (сила волны)	модуль комплексного числа, делённый на N	<code>np.abs(X) / N</code>
Фаза (сдвиг)	угол комплексного числа	<code>np.angle(X)</code>

Из-за ошибок округления "ненужные" частоты получаются не ровно 0, а очень маленькими числами. Поэтому мы задаём порог `threshold` и считаем значимыми только те частоты, где амплитуда больше него.

Что считать шириной спектра. Это диапазон частот, в котором есть сигнал. Есть два способа считать:

- `max - min` - от самой низкой до самой высокой значимой частоты (у нас 4 и 8 Гц, получится 4 Гц). Так считают для полосовых сигналов, у которых спектр где-то посередине.
- `max` - от нуля до самой высокой частоты (получится 8 Гц). Так считают для сигналов в основной полосе, которые мы оцифровываем с нуля.

Я взял второй вариант: $f_{\max} = 8$ Гц.

```
1 y_dft = discrete_fourier_transform(y_sampled)
2 N = len(y_dft)
3 freqs = np.arange(N) * (min_freq_kotelnikov / N)
4 amp = np.abs(y_dft) / N
5
6 ic(freqs)
7 ic(amp)
8
9 threshold = 1e-6 # порог для выделения значимых частот
10 half = N // 2 + 1 # без зеркальной части
11 significant = np.array([freqs[k] for k in range(half) if amp[k] >
    threshold])
12 # spectrum_width = significant.max() - significant.min()
13 spectrum_width = significant.max()
14
15 ic(spectrum_width);
```

Листинг 1.14 - Прямое ДПФ и оценка ширины спектра

```
ic| freqs: array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.,
    10., 11., 12.,
    13., 14., 15., 16., 17., 18., 19., 20., 21., 22.,
    23., 24., 25.,
    26., 27., 28., 29., 30., 31.])
ic| amp: array([2.77555756e-17, 7.39993399e-17, 3.16461231e-17,
    3.51614705e-16,
    5.00000000e-01, 3.94695830e-16, 1.97240692e-16,
    1.54996458e-16,
    5.00000000e-01, 1.13698104e-16, 2.95841644e-16,
    2.62533797e-16,
    3.64355492e-16, 8.13652846e-16, 5.08091498e-16,
    3.89364649e-16,
    1.83568042e-15, 6.64082938e-16, 6.87560229e-16,
    4.47316182e-16,
    3.45556987e-16, 3.87944218e-16, 3.95369749e-16,
    1.23716373e-15,
    5.00000000e-01, 6.70588694e-16, 5.00919297e-16,
    1.54605159e-15,
    5.00000000e-01, 1.29373207e-15, 6.35261403e-16,
    3.62091088e-16])
ic| spectrum_width: np.float64(8.0)
```

Листинг 1.15 - Результат выполнения: прямое ДПФ и оценка ширины спектра

```

1 show_plot(
2     t=freqs,
3     func_val=amp,
4     is_stem=True,
5     title="Амплитудный спектр, все N гармоник зеркальная ( часть )",
6 )

```

Листинг 1.16 - Построение полного амплитудного спектра



Рисунок 4 - Амплитудный спектр, все N гармоник (с зеркальной частью)

```

1 def draw_threshold(ax: Axes) -> None:
2     ax.axhline(threshold, color="gray", linestyle="--", label="порог")
3     ax.axvline(spectrum_width, color="r", linestyle=":", label="ширина
4     спектра")
5     ax.legend()
6 show_plot(
7     t=freqs[:half],
8     func_val=amp[:half],
9     is_stem=True,
10    title="Амплитудный спектр без зеркальной части",
11    on_axes=draw_threshold,
12 )

```

Листинг 1.17 - Построение спектра без зеркальной части



Рисунок 5 - Амплитудный спектр без зеркальной части

Объём памяти для `y_sampled`

```
1 ic(y_sampled.dtype)    # float64 - 64 бита / 8 байт на отсчёт
2 ic(y_sampled.size)     # 32 отсчёта (fs = 32 Гц на 1 с)
3 ic(y_sampled.nbytes);  # 256 байт = 32 * 8
```

Листинг 1.18 - Объём памяти массива отсчётов

```
ic| y_sampled.dtype: dtype('float64')
ic| y_sampled.size: 32
ic| y_sampled.nbytes: 256
```

Листинг 1.19 - Результат выполнения: объём памяти массива отсчётов

Объём памяти для `y_dft`

```
1 ic(y_dft.dtype)        # complex128 - 128 бит / 16 байт на отсчёт
2 ic(y_dft.size)         # 32 отсчёта (fs = 32 Гц на 1 с)
3 ic(y_dft.nbytes);      # 512 байт = 32 * 16
```

Листинг 1.20 - Объём памяти массива спектра

```
ic| y_dft.dtype: dtype('complex128')
ic| y_dft.size: 32
ic| y_dft.nbytes: 512
```

Листинг 1.21 - Результат выполнения: объём памяти массива спектра

Вывод

- В спектре две гармоники: на 4 Гц и на 8 Гц.
- Остальные частоты равны нулю (с точностью до ошибок округления), поэтому ширина спектра равна 8 Гц, как и максимальная частота из задания 2.
- Для хранения 32 отсчётов в `float64` нужно 256 байт, для их спектра в `complex128` - 512 байт.

1.4.6 Задание 6

Восстановите оригинальный аналоговый сигнал по массиву имеющихся у вас отсчетов, соединив их непрерывной линией и оцените визуальное сходство оригинального сигнала и восстановленного после оцифровки.

```
1 show_multi_plot(
2     t=[t, t_sampled],
3     func_vals=[y, y_sampled],
4     labels=["оригинал", "восстановленный"],
5     title=f"Восстановление по отсчётам, fs = {min_freq_kotelnikov}",
6 )
```

Листинг 1.22 - Восстановление сигнала по отсчётам



Рисунок 6 - Восстановление сигнала по отсчётам, $f_s = 32$ Гц

```

1 y_restored = np.interp(t, t_sampled, y_sampled)
2 mask = t <= t_sampled[-1] # только там, где есть отсчёты с обеих
  сторон
3 restore_error = np.mean(np.abs(y[mask] - y_restored[mask]))
4
5 ic(restore_error);

```

Листинг 1.23 - Ошибка восстановления

```
ic| restore_error: np.float64(0.13764728860662287)
```

Листинг 1.24 - Результат выполнения: ошибка восстановления

Вывод

Восстановленный сигнал повторяет форму оригинала, но в виде ломаной линии, потому что на период самой быстрой гармоники (8 Гц) приходится всего 4 отсчёта. Частота по Котельникову гарантирует, что информация о сигнале сохранена, но простое соединение точек прямыми восстанавливает его грубо.

1.4.7 Задание 7

Увеличьте частоту дискретизации в 4 раза и проделайте задания из п.4-6.

Пункт 4 ($fs \times 4$)

```
1 min_freq_4x = min_freq_kotelnikov * 4
2 step_4x = len(y) // int(min_freq_4x * (stop - start))
3
4 y_sampled_4x = y[::step_4x]
5 t_sampled_4x = t[::step_4x]
6
7 ic(min_freq_4x)
8 ic(step_4x)
9 ic(y_sampled_4x.size);
```

Листинг 1.25 - Дискретизация с частотой, увеличенной в 4 раза

```
ic| min_freq_4x: 128
ic| step_4x: 8
ic| y_sampled_4x.size: 128
```

Листинг 1.26 - Результат выполнения: дискретизация с частотой, увеличенной в 4 раза

```
1 def draw_samples_4x(ax: Axes) -> None:
2     ax.stem(t_sampled_4x, y_sampled_4x, linefmt="r-", markerfmt="ro",
3           basefmt=" ")
4
5 show_plot(
6     t=t,
7     func_val=y,
8     title="Сигнал и отсчеты,  $fs \times 4$ ",
9     on_axes=draw_samples_4x,
```

Листинг 1.27 - Построение сигнала и отсчётов при увеличенной в 4 раза частоте

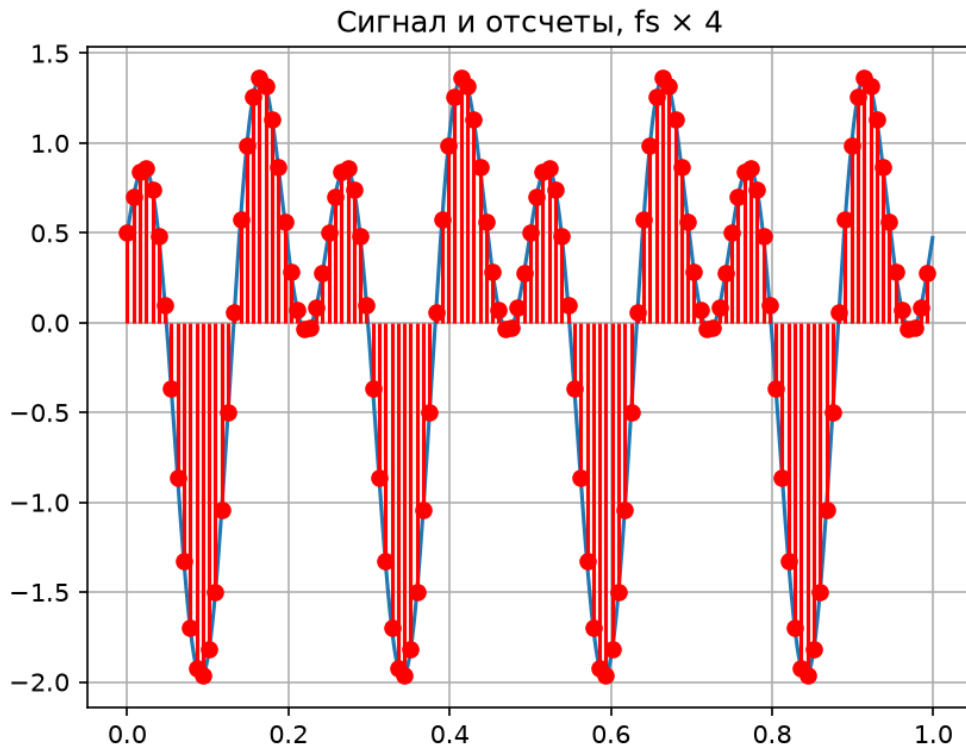


Рисунок 7 - Сигнал и отсчеты, $f_s = 128$ Гц

Пункт 5 ($f_s \times 4$)

```

1 y_dft_4x = discrete_fourier_transform(y_sampled_4x)
2 N_4x = len(y_dft_4x)
3 freqs_4x = np.arange(N_4x) * (min_freq_4x / N_4x)
4 amp_4x = np.abs(y_dft_4x) / N_4x
5
6 half_4x = N_4x // 2 + 1 # без зеркальной части
7 significant_4x = np.array([freqs_4x[k] for k in range(half_4x) if
8     amp_4x[k] > threshold])
9 spectrum_width_4x = significant_4x.max()
10 ic(spectrum_width_4x)
11 ic(y_sampled_4x.nbytes);

```

Листинг 1.28 - ДПФ и ширина спектра при увеличенной в 4 раза частоте

```

ic| spectrum_width_4x: np.float64(8.0)
ic| y_sampled_4x.nbytes: 1024

```

Листинг 1.29 - Результат выполнения: дПФ и ширина спектра при увеличенной в 4 раза частоте

```

1 def draw_threshold_4x(ax: Axes) -> None:
2     ax.axhline(threshold, color="gray", linestyle="--", label="порог")

```

```

3     ax.axvline(spectrum_width_4x, color="r", linestyle=":", label="
    ширина спектра")
4     ax.legend()
5
6 show_plot(
7     t=freqs_4x[:half_4x],
8     func_val=amp_4x[:half_4x],
9     is_stem=True,
10    title="Амплитудный спектр без зеркальной части,  $f_s \times 4$ ",
11    on_axes=draw_threshold_4x,
12 )

```

Листинг 1.30 - Спектр при увеличенной в 4 раза частоте



Рисунок 8 - Амплитудный спектр без зеркальной части, $f_s = 128$ Гц

Пункт 6 ($f_s \times 4$)

```

1 show_multi_plot(
2     t=[t, t_sampled_4x],
3     func_vals=[y, y_sampled_4x],
4     labels=["оригинал", "восстановленный"],
5     title=f"Восстановление по отсчётам,  $f_s = \{min\_freq\_4x\}$ ",
6 )

```

Листинг 1.31 - Восстановление по отсчётам при увеличенной в 4 раза частоте



Рисунок 9 - Восстановление сигнала по отсчётам, $f_s = 128$ Гц

```

1 y_restored_4x = np.interp(t, t_sampled_4x, y_sampled_4x)
2 mask_4x = t <= t_sampled_4x[-1] # только там, где есть отсчёты с
  обеих сторон
3 restore_error_4x = np.mean(np.abs(y[mask_4x] - y_restored_4x[mask_4x]))
4
5 ic(restore_error_4x);

```

Листинг 1.32 - Ошибка восстановления при увеличенной в 4 раза частоте

```
ic| restore_error_4x: np.float64(0.008166292192515932)
```

Листинг 1.33 - Результат выполнения: ошибка восстановления при увеличенной в 4 раза частоте

Что изменилось

- Отсчётов стало в 4 раза больше, поэтому и памяти нужно в 4 раза больше.
- Восстановленная линия стала гораздо ближе к оригиналу.

1.4.8 Задание 8

Запишите аудиофайл со своим голосом (например, в формате wav). Проанализируйте визуально спектр голоса, используя, например, приложение Audacity (вкладки Анализ → График спектра). Определите максимальную частоту в спектре данного сигнала и выберите требуемую для оцифровки частоту дискретизации.

Запись голоса `voice.wav` (6,14 с, 44100 Гц, моно).

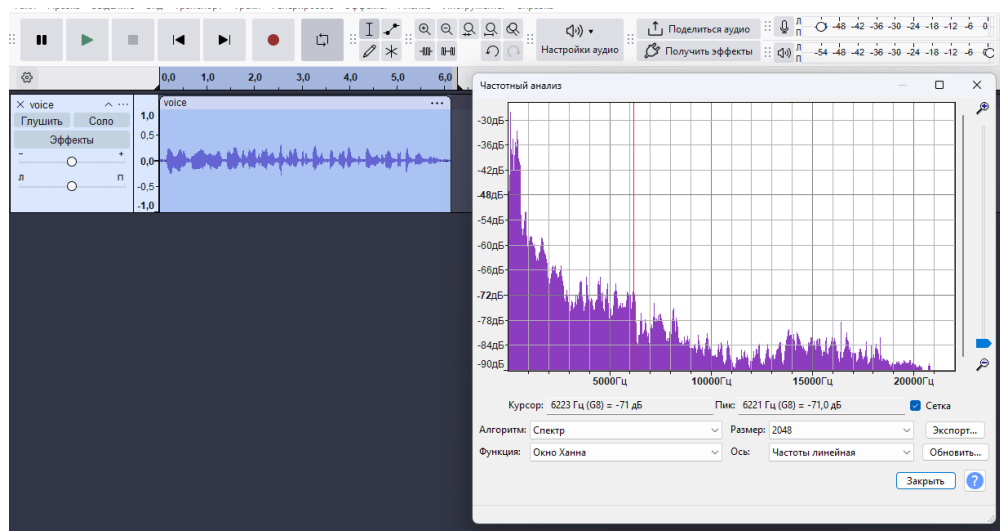


Рисунок 10 - Временное и частотное представление голоса в Audacity

Максимальная частота и частота дискретизации

На графике видно, что почти вся энергия голоса сосредоточена в низких частотах, а дальше спектр спадает. Заметный голосовой спектр заканчивается примерно на частоте $f_{max} \approx 6200$ Гц (курсор на: 6223 Гц, -71 дБ). Дальше идёт уровень шума, там самого голоса уже нет.

По теореме Котельникова частота дискретизации должна быть строго больше удвоенной максимальной частоты:

$$f_s > 2f_{max} = 2 \cdot 6200 = 12400 \text{ Гц}$$

Значит, для оцифровки такого голоса достаточно, например, $f_s = 16000$ Гц.

1.4.9 Задание 9

Используя библиотеки работы со звуком Matlab (или Python) проанализируйте имеющуюся у вас запись голоса. Считайте аудиофайл с помощью функции Matlab (файл с голосом должен лежать в рабочей директории Matlab):

```
1 [y, Fs] = audioread('voice.wav');
```

где y - это массив отсчетов голоса, а F_s - частота дискретизации или семплирования, которая определяется в Matlab для данного файла.

```
1 Fs, y_voice = wavfile.read("voice.wav")
2
3 if y_voice.ndim > 1:
4     y_voice = y_voice[:, 0]
5
6 if np.issubdtype(y_voice.dtype, np.integer):
7     y_voice = y_voice / np.iinfo(y_voice.dtype).max # приводим к
    диапазону -1..1
8
9 ic(Fs)
10 ic(y_voice.size);
```

Листинг 1.34 - Чтение файла voice.wav

```
ic| Fs: 44100
ic| y_voice.size: 270848
```

Листинг 1.35 - Результат выполнения: чтение файла voice.wav

1.4.10 Задание 10

Определите частоту дискретизации, которая была использована при записи голоса на цифровой носитель. Для этого возьмите количество элементов в файле с записью и разделите на длительность записи в секундах. Например, если у вас файл состоит из 441 000 элементов (отсчетов) и имеет длительность 10 секунд, то частота дискретизации F_s будет равна $441\,000/10 = 44\,100$ Гц. Сравните вычисленное значение с тем, что выводится в Matlab для F_s .

```
1 # duration = y_voice.size / Fs
2 duration = 6.141678004535147
```

```

3
4 Fs_calc = y_voice.size / duration
5
6 ic(duration)
7 ic(Fs_calc)
8 ic(Fs);

```

Листинг 1.36 - Расчёт частоты дискретизации записи

```

ic| duration: 6.141678004535147
ic| Fs_calc: 44100.0
ic| Fs: 44100

```

Листинг 1.37 - Результат выполнения: расчёт частоты дискретизации записи

Вывод

Частота дискретизации, посчитанная как число отсчётов, делённое на длительность, совпала с той, что вернула `wavfile.read`, и равна 44100 Гц.

1.4.11 Задание 11

Прорежьте полученный массив `y` с помощью функции `downsample` (иными словами - уменьшите частоту дискретизации) и воспроизведите полученный сигнал с помощью `matlab`-функции `play()` или ее аналогами в других языках программирования. Обратите внимание на качество звучания при неверно взятой частоте дискретизации:

```

1 y1=downsample(y, 10); % прореживаем массив y, оставляя лишь каждый 10
   отсчет
2 zvuk = audioplayer(y1,Fs/10); %создаем объект gong типа audioplayer
3 play(zvuk); %воспроизводим звук
4 plot(y1); %визуализируем амплитуду сигнала на графике в зависимости от
   времени

```

Чтобы остановить воспроизведение звука, введите `stop(zvuk)` в командную строку.

```

1 Fs_down = Fs // 10
2 y_voice_down = y_voice[:,10]
3
4 ic(Fs_down)
5 ic(y_voice_down.size);

```

Листинг 1.38 - Прореживание записи

```
ic| Fs_down: 4410  
ic| y_voice_down.size: 27085
```

Листинг 1.39 - Результат выполнения: прореживание записи

Оригинал

```
1 ic(Fs)  
2 Audio(y_voice, rate=Fs)
```

Листинг 1.40 - Воспроизведение оригинала

```
ic| Fs: 44100
```

Листинг 1.41 - Результат выполнения: воспроизведение оригинала

Прореженный сигнал

```
1 ic(Fs_down)  
2 Audio(y_voice_down, rate=Fs_down)
```

Листинг 1.42 - Воспроизведение прореженного сигнала

```
ic| Fs_down: 4410
```

Листинг 1.43 - Результат выполнения: воспроизведение прореженного сигнала

```
1 t_voice_down = np.arange(y_voice_down.size) / Fs_down  
2  
3 show_plot(  
4     t=t_voice_down,  
5     func_val=y_voice_down,  
6     title="Голос, каждый й10- отсчёт",  
7 )
```

Листинг 1.44 - Построение прореженного сигнала

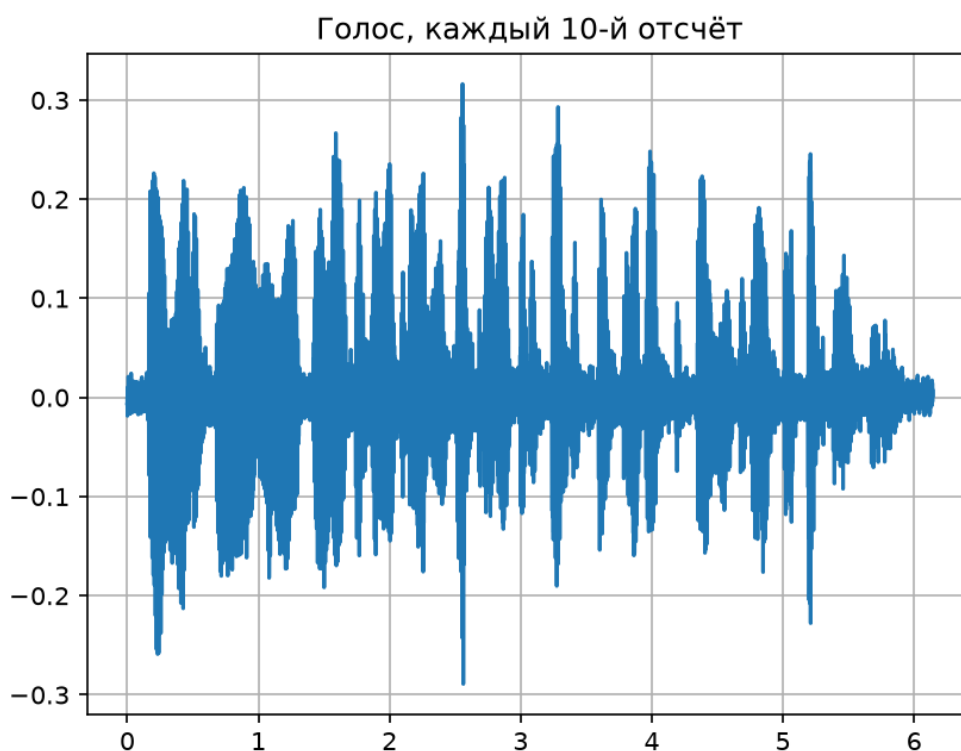


Рисунок 11 - Прореженный сигнал голоса (каждый 10-й отсчёт)

Вывод

Оригинал звучит чисто. Прореженный сигнал звучит глухо, с искажениями: мы оставили только 4410 отсчётов в секунду, а голосу нужно хотя бы 14 кГц (по теореме Котельникова). Это и есть последствия неверно выбранной частоты дискретизации.

1.4.12 Задание 12

Выполните прямое дискретное преобразование Фурье для оригинального звучания и для прореженного сигнала, выведите на график амплитудный спектр сигнала, определите его ширину.

`discrete_fourier_transform` считает за N^2 операций, а у записи сотни тысяч отсчётов. Поэтому для ускорения я взял готовую реализацию быстрого преобразования Фурье `np.fft.fft`.

Ширину спектра у голоса ровно до нуля не найти, поэтому считаем значимыми частоты, где амплитуда больше 1% от самой большой.

```
1 def get_spectrum(x: np.ndarray, fs: float) -> tuple[np.ndarray, np.
   ndarray]:
2     N = len(x)
```

```

3     # X = discrete_fourier_transform(x)
4     X = np.fft.fft(x)
5     half = N // 2 + 1 # без зеркальной части
6
7     return np.arange(half) * (fs / N), np.abs(X[:half]) / N
8
9
10 def get_spectrum_width(freqs: np.ndarray, amp: np.ndarray, level: float
    = 0.01) -> float:
11     return freqs[amp > level * amp.max()].max()

```

Листинг 1.45 - Функции расчёта спектра и его ширины

```

1 freqs_voice, amp_voice = get_spectrum(y_voice, Fs)
2 freqs_voice_down, amp_voice_down = get_spectrum(y_voice_down, Fs_down)
3
4 width_voice = get_spectrum_width(freqs_voice, amp_voice)
5 width_voice_down = get_spectrum_width(freqs_voice_down, amp_voice_down)
6
7 ic(width_voice)
8 ic(width_voice_down);

```

Листинг 1.46 - Спектры и ширина спектров голоса

```

ic| width_voice: np.float64(7057.02903473535)
ic| width_voice_down: np.float64(2204.7557688757615)

```

Листинг 1.47 - Результат выполнения: спектры и ширина спектров голоса

```

1 def draw_width_voice(ax: Axes) -> None:
2     ax.axvline(width_voice, color="r", linestyle=":", label="ширина
    спектра")
3     ax.legend()
4
5 show_plot(
6     t=freqs_voice,
7     func_val=amp_voice,
8     title="Амплитудный спектр голоса, оригинал",
9     on_axes=draw_width_voice,
10 )

```

Листинг 1.48 - Построение спектра голоса (оригинал)



Рисунок 12 - Амплитудный спектр голоса, оригинал

```

1 show_plot(
2     t=freqs_voice[:len(freqs_voice)//20],
3     func_val=amp_voice[:len(amp_voice)//20],
4     title="Амплитудный спектр голоса, оригинал до (й20- части)"
5 )

```

Листинг 1.49 - Построение спектра голоса (участок)



Рисунок 13 - Амплитудный спектр голоса, оригинал (первая двадцатая часть)

```

1 def draw_width_voice_down(ax: Axes) -> None:
2     ax.axvline(width_voice_down, color="r", linestyle=":", label="
    ширина спектра")
3     ax.legend()
4
5 show_plot(
6     t=freqs_voice_down,
7     func_val=amp_voice_down,
8     title="Амплитудный спектр голоса, каждый й10- отсчёт",
9     on_axes=draw_width_voice_down,
10 )

```

Листинг 1.50 - Построение спектра голоса (прореженный сигнал)



Рисунок 14 - Амплитудный спектр голоса, каждый 10-й отсчёт

Вывод

- Основная энергия голоса лежит в самых низких частотах (до нескольких сотен герц), дальше амплитуды спектра быстро падают.
- У оригинала (44100 Гц) ширина спектра около 7 кГц, это согласуется с Audacity (задание 8).
- После прореживания в 10 раз частота дискретизации стала 4410 Гц, и спектр обрезан на 2205 Гц (половина от новой частоты). Всё, что в голосе было выше, потерялось или "сложилось" с нижними частотами, поэтому звук стал глухим и "шакальным".

1.4.13 Задание 13

Оцените влияние разрядности АЦП на спектр сигнала. Для этого нужно написать функцию, которая бы округляла значения отсчетов сигнала, заданного в вашем варианте, до какого-то числа, определяемого разрядностью АЦП. Допустим если у АЦП всего 3 разряда, то диапазон возможных дискретных значений амплитуд временных отсчетов сигнала - это 0..7 (то есть, $7 = 2^3 - 1$). Все значения больше 7 округляются до 7. Для результирующе-

го дискретного сигнала требуется выполнить прямое преобразование Фурье. Сравнить полученный спектр со спектром исходной синусоиды, отсчеты которой не подвергались квантованию по уровню. Вывести среднюю ошибку квантования для случаев, когда разрядность АЦП равна 3/4/5/6.

Как работает квантование

У АЦП с `bits` разрядами есть всего 2^{bits} уровней (коды $0 \dots 2^{\text{bits}} - 1$). Наш сигнал бывает и отрицательным, поэтому сначала мы сдвигаем и растягиваем его так, чтобы он занял диапазон $0 \dots 2^{\text{bits}} - 1$, округляем до целого (это и есть код АЦП), всё, что вылезло за диапазон, обрезаем, и возвращаем обратно в исходный масштаб, чтобы можно было сравнивать с оригиналом.

Ошибка квантования - разница между исходным отсчётом и тем, что получилось после округления. Средняя ошибка - среднее модуля этой разницы.

```
1 bits_list = [3, 4, 5, 6]
2 quantized = {bits: quantize(y_sampled, bits) for bits in bits_list}
3 quant_errors = {bits: np.mean(np.abs(y_sampled - quantized[bits])) for
4                   bits in bits_list}
5
6 ic(y_sampled.max())
7 ic(y_sampled.min())
8 ic("-----")
9
10 for bits in bits_list:
11     ic(bits)
12     ic(quant_errors[bits])
13     ic("-----")
```

Листинг 1.51 - Квантование и средняя ошибка квантования

```
ic| y_sampled.max(): np.float64(1.2588190451025218)
ic| y_sampled.min(): np.float64(-1.965925826289069)
ic| '-----'
ic| bits: 3
ic| quant_errors[bits]: np.float64(0.07986980422058564)
ic| '-----'
ic| bits: 4
ic| quant_errors[bits]: np.float64(0.036611652351682206)
ic| '-----'
ic| bits: 5
```

```

ic| quant_errors[bits]: np.float64(0.019374196344376298)
ic| '-----'
ic| bits: 6
ic| quant_errors[bits]: np.float64(0.011018439086669527)
ic| '-----'

```

Листинг 1.52 - Результат выполнения: квантование и средняя ошибка квантования

Спектры после квантования

На каждом графике палочки - спектр квантованного сигнала, красные точки - спектр исходного.

```

1 def get_half_amp(x: np.ndarray) -> np.ndarray:
2     return np.abs(discrete_fourier_transform(x))[:half] / len(x)
3
4 amp_origin = get_half_amp(y_sampled)
5
6 for bits in bits_list:
7     def draw_origin(ax: Axes) -> None:
8         ax.plot(freqs[:half], amp_origin, "ro", label="без квантования"
9             )
10        ax.legend()
11
12    show_plot(
13        t=freqs[:half],
14        func_val=get_half_amp(quantized[bits]),
15        is_stem=True,
16        title=f"Спектр, АЦП {bits} бит",
17        on_axes=draw_origin,
18    )

```

Листинг 1.53 - Спектры после квантования

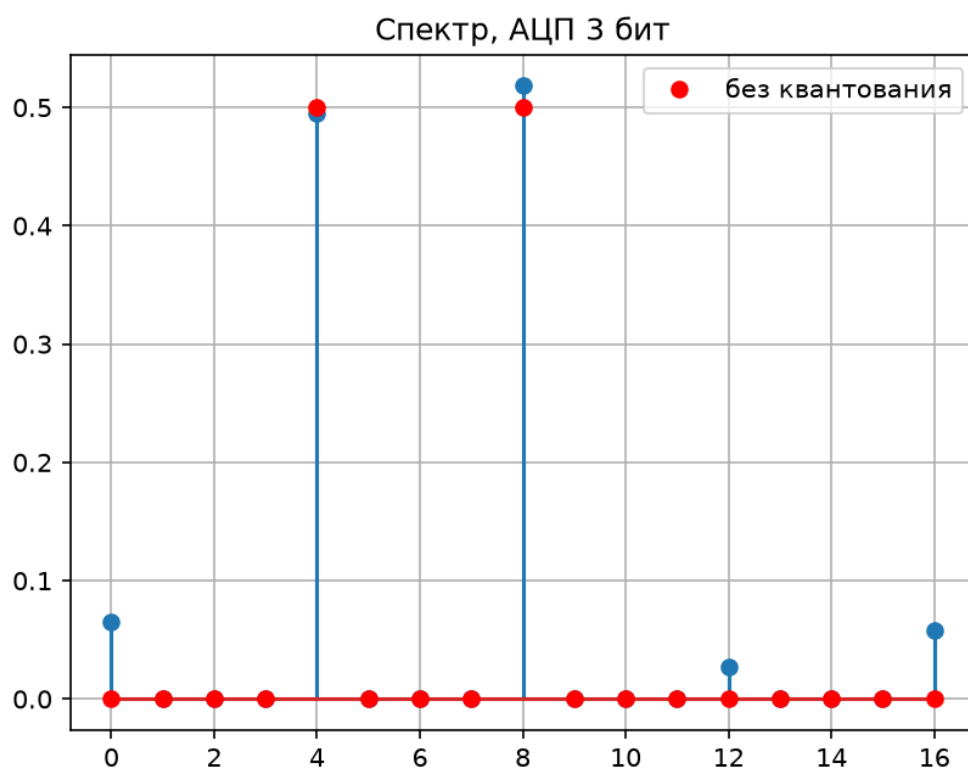


Рисунок 15 - Спектр квантованного сигнала, АЦП 3 бит

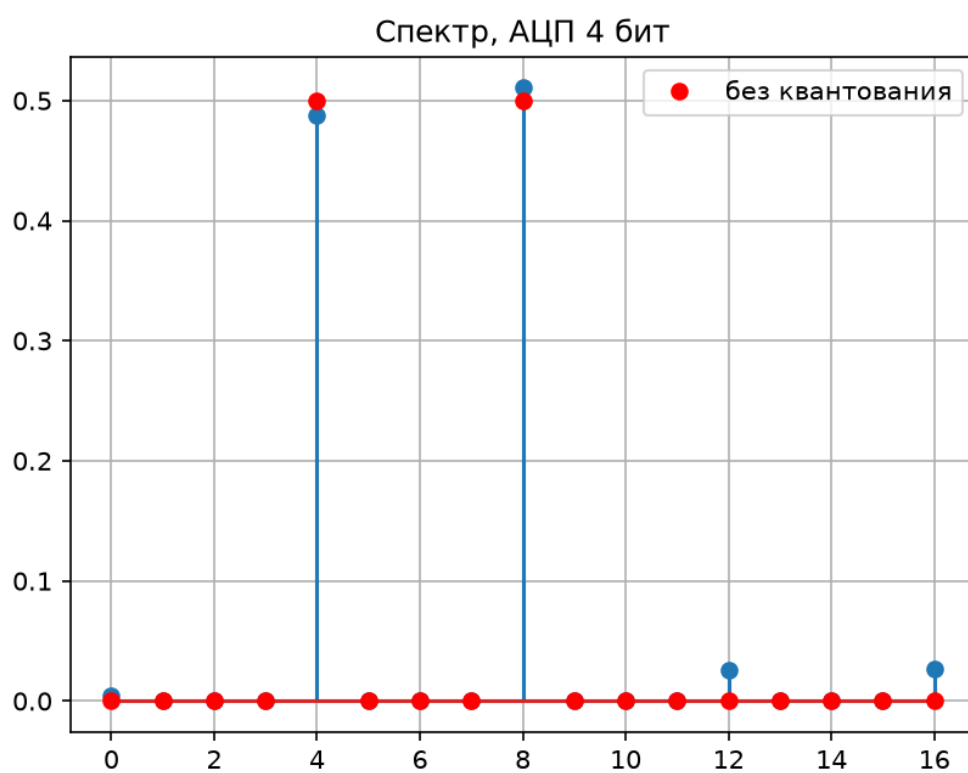


Рисунок 16 - Спектр квантованного сигнала, АЦП 4 бит

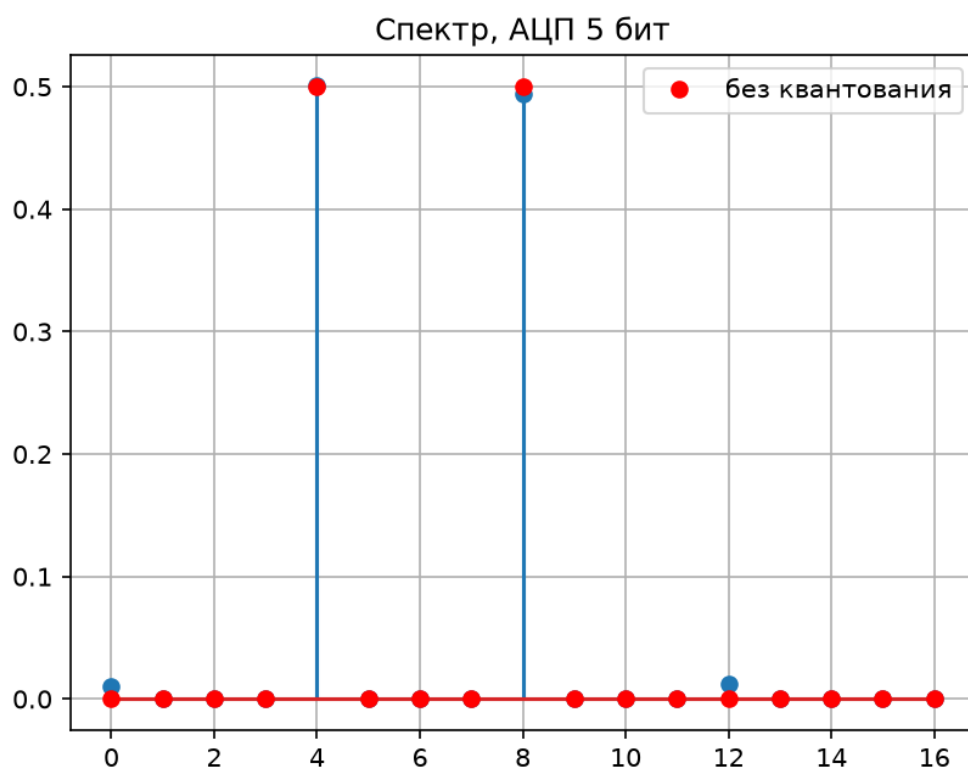


Рисунок 17 - Спектр квантованного сигнала, АЦП 5 бит

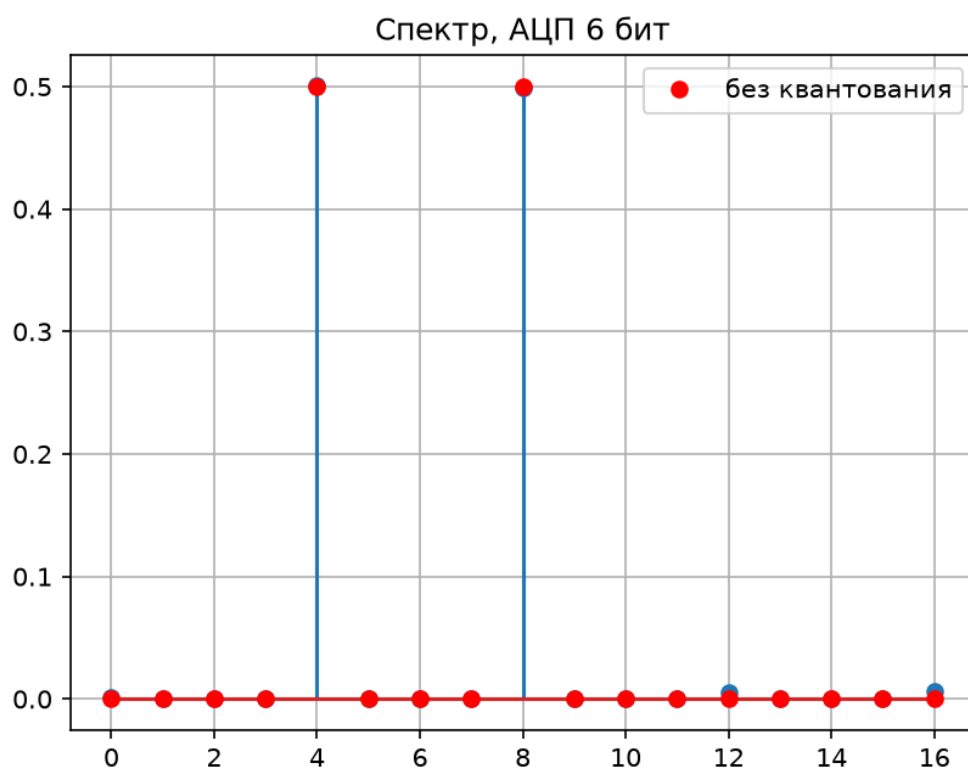


Рисунок 18 - Спектр квантованного сигнала, АЦП 6 бит

Вывод

Чем больше разрядов у АЦП, тем меньше средняя ошибка квантования (каждый лишний разряд примерно вдвое уменьшает её). На спектре ошибка выглядит как "шум" - маленькие палочки на частотах, где в исходном сигнале ничего нет. С ростом разрядности этот шум падает, а основные пики (4 и 8 Гц) почти не меняются.

1.5 Ответы на контрольные вопросы

1) Для чего используются прямое и обратное преобразование Фурье?

Прямое преобразование Фурье переводит сигнал из временного представления в частотное: показывает, из каких гармоник он состоит и какие у них амплитуды и фазы. Обратное делает наоборот: по частотному представлению восстанавливает сигнал во времени. Вместе они позволяют анализировать спектр сигнала, фильтровать его в частотной области и возвращаться обратно.

2) Что такое ошибка квантования и дискретизации?

Ошибка дискретизации возникает из-за того, что мы берём сигнал только в отдельные моменты времени. Если частота дискретизации слишком мала (не выполняется теорема Котельникова), часть информации теряется, а высокие частоты "складываются" с низкими. Ошибка квантования возникает из-за того, что амплитуду отсчёта приходится округлять до одного из 2^n уровней АЦП. Она не превышает половины шага квантования и тем меньше, чем больше разрядов у АЦП.

3) Какое количество разрядов АЦП требуется, чтобы оцифровать голос?

Для разборчивой речи (телефонная связь) хватает 8 разрядов. Для качественной записи обычно берут 16 разрядов: это 65536 уровней и динамический диапазон около 96 дБ.

4) Как математически получить дискретные отсчеты непрерывного сигнала?

Нужно взять значения непрерывной функции в моменты времени, кратные периоду дискретизации $T_s = 1/f_s$:

$$x[n] = x(nT_s) = x\left(\frac{n}{f_s}\right), \quad n = 0, 1, 2, \dots$$

В коде это то же самое, что делать срез `y[: : step]`.

5) Какой спектр у периодического сигнала $\sin(10\pi t + \pi/2)$?

Так как $\sin(10\pi t + \pi/2) = \sin(2\pi \cdot 5 \cdot t + \pi/2) = \cos(10\pi t)$, это одна гармоника с частотой $f = 5$ Гц и амплитудой 1. Спектр состоит из одной линии на 5 Гц (в полном спектре ещё зеркальная линия на -5 Гц, каждая с амплитудой 0,5), начальная фаза $\pi/2$ у синусной формы записи.

6) Что такое быстрое преобразование Фурье?

Это алгоритм, который считает то же дискретное преобразование Фурье, но быстрее. Он использует повторяющиеся значения синуса и косинуса и группирует слагаемые с одинаковыми множителями. Вместо порядка N^2 операций нужно порядка $N \log_2 N$, а длина входа обычно равна степени двойки ($N = 2^k$). Поэтому БПФ используют в реальных модемах.

7) Как определяется минимальная требуемая для оцифровки частота дискретизации сигнала?

По теореме Котельникова: частота дискретизации должна быть строго больше удвоенной максимальной частоты в спектре сигнала,

$$f_s > 2f_{max}.$$

1.6 Вывод

В работе сгенерирован непрерывный сигнал варианта 5, представляющий собой сумму двух гармоник с частотами 4 и 8 Гц, поэтому максимальная частота его спектра равна 8 Гц. По теореме Котельникова частота дискретизации должна быть строго больше 16 Гц, при ровно 16 Гц отсчёты восьмigerцовой гармоники попадают в нули синуса и гармоника теряется. Выбрана частота 32 Гц: она в 4 раза больше максимальной частоты и нацело делит сетку из 1024 точек, поэтому отсчёты получаются равномерными.

Прямое дискретное преобразование Фурье для 32 отсчётов показало две значимые гармоники на частотах 4 и 8 Гц с амплитудой 0,5 каждая (вторая половина спектра - зеркальное отражение), амплитуды остальных частот

порядка 10^{-16} и объясняются ошибками округления. Ширина спектра составила 8 Гц. Массив отсчётов в формате `float64` занимает 256 байт, массив спектра в формате `complex128` - 512 байт.

При соединении отсчётов линией восстановленный сигнал повторяет форму оригинала, но имеет вид ломаной, так как на период самой быстрой гармоники приходится всего 4 отсчёта, средняя ошибка составила 0,138. Увеличение частоты дискретизации в 4 раза (до 128 Гц) не изменило ширину спектра (8 Гц), так как новых частот в сигнале не появилось, но увеличило объём памяти в 4 раза (1024 байта) и уменьшило среднюю ошибку восстановления до 0,0082.

Анализ записи голоса в Audacity показал, что основная энергия сосредоточена в низких частотах, максимальная частота голоса около 6,2 кГц, следовательно, для оцифровки достаточно частоты дискретизации выше 12,4 кГц, например 16 кГц. Частота дискретизации записи, рассчитанная как число отсчётов, делённое на длительность, совпала со значением из файла и составила 44100 Гц. Ширина спектра оригинала при пороге 1% от максимальной амплитуды около 7,06 кГц. После прореживания записи в 10 раз частота дискретизации стала 4410 Гц, спектр обрезан на частоте 2205 Гц, часть высоких частот потеряна или наложилась на низкие, поэтому звук стал глухим и искажённым.

Оценка влияния разрядности АЦП показала, что средняя ошибка квантования для 3, 4, 5 и 6 разрядов составляет 0,080, 0,037, 0,019 и 0,011 соответственно, то есть каждый дополнительный разряд уменьшает её примерно вдвое. На спектре ошибка квантования проявляется как шум малой амплитуды на частотах, которых нет в исходном сигнале, с ростом разрядности он уменьшается, а основные пики на частотах 4 и 8 Гц почти не меняются.

1.7 Ссылка на репозиторий с кодом

Исходный код работы доступен в репозитории: <https://github.com/Emeteil/mobile-communications-basics>.



Рисунок 19 - QR-код со ссылкой на репозиторий с кодом